MUnit

Link: https://mulesy.com/category/3-mulesoft-working-samples/munits/

>> using Munit we can write unit test cases and functional test cases.
>> for performance testing we cannot use Munit.

Munit is of 2 type:

1. manual
2. Record

>> **Record had limitations:**

The test recorder has the following limitations when you create MUnit tests:

- You cannot create tests for flows with Mule errors raised inside the flow or already existing in the incoming event, even if an on error continue error handler controls them.

- A recorded flow execution ends successfully, but the result does not reach its destination because the application is killed.

- Your validations fail every time your test runs if you configure spy or assert processors to assert values for random data, time-dependent information (such as timestamps), or values resulting from parallel processes, because those values change in every execution.

- Mocking values resulting from parallel processes causes a mixture of real and mocked data that compromises the execution of the processors that follow in your test.

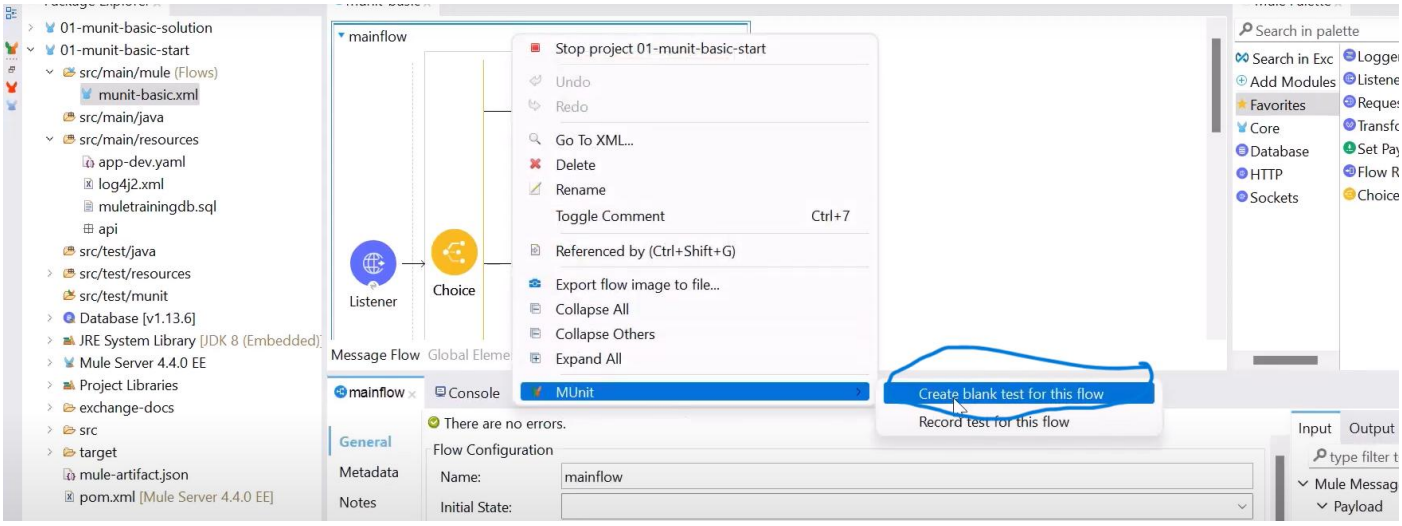
- Although the recorder supports data iteration in the flow, such as recursions or loops, it does not support cases in which the structure of the data being tested changes inside the iteration.

- The recorder does not support mocking a message before or inside a foreach processor.

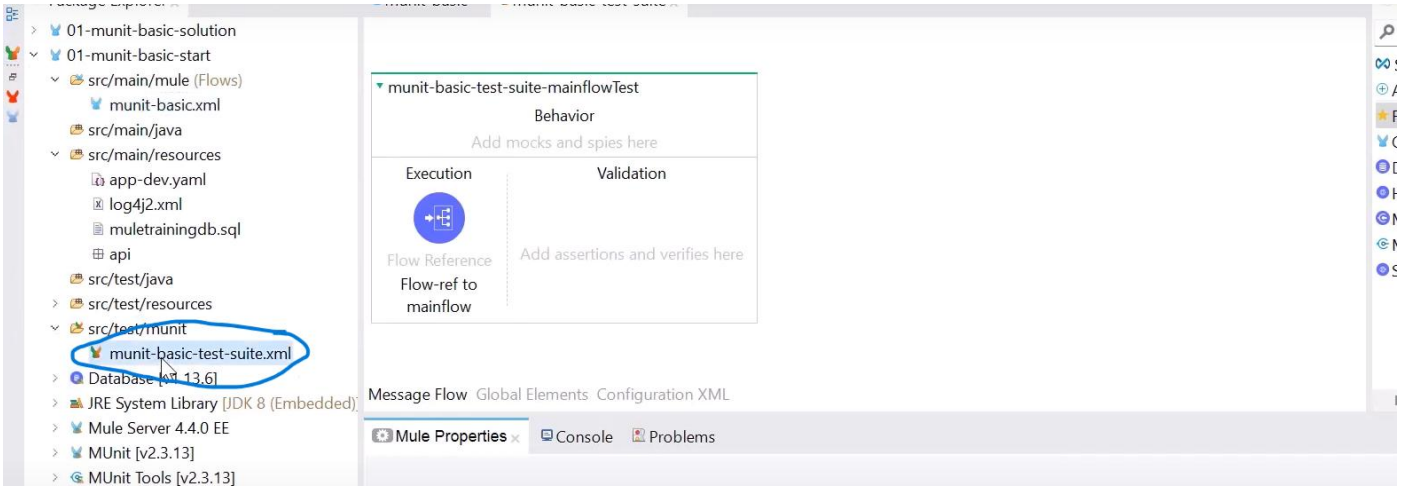
- When using the Test Recorder to generate a test over your flow, you can only mock parameters included in your flow. You cannot mock parameters in sub-flows or in referenced flows through the Test Recorder UI. If you need to mock parameters in sub-flows of any other process in the project, you can modify the mocking component after creating the test.

1. Manual test:

To create test cases, right click on main flow > Munit > create blank test



It will create test suite., it is a combination of multiple test cases.



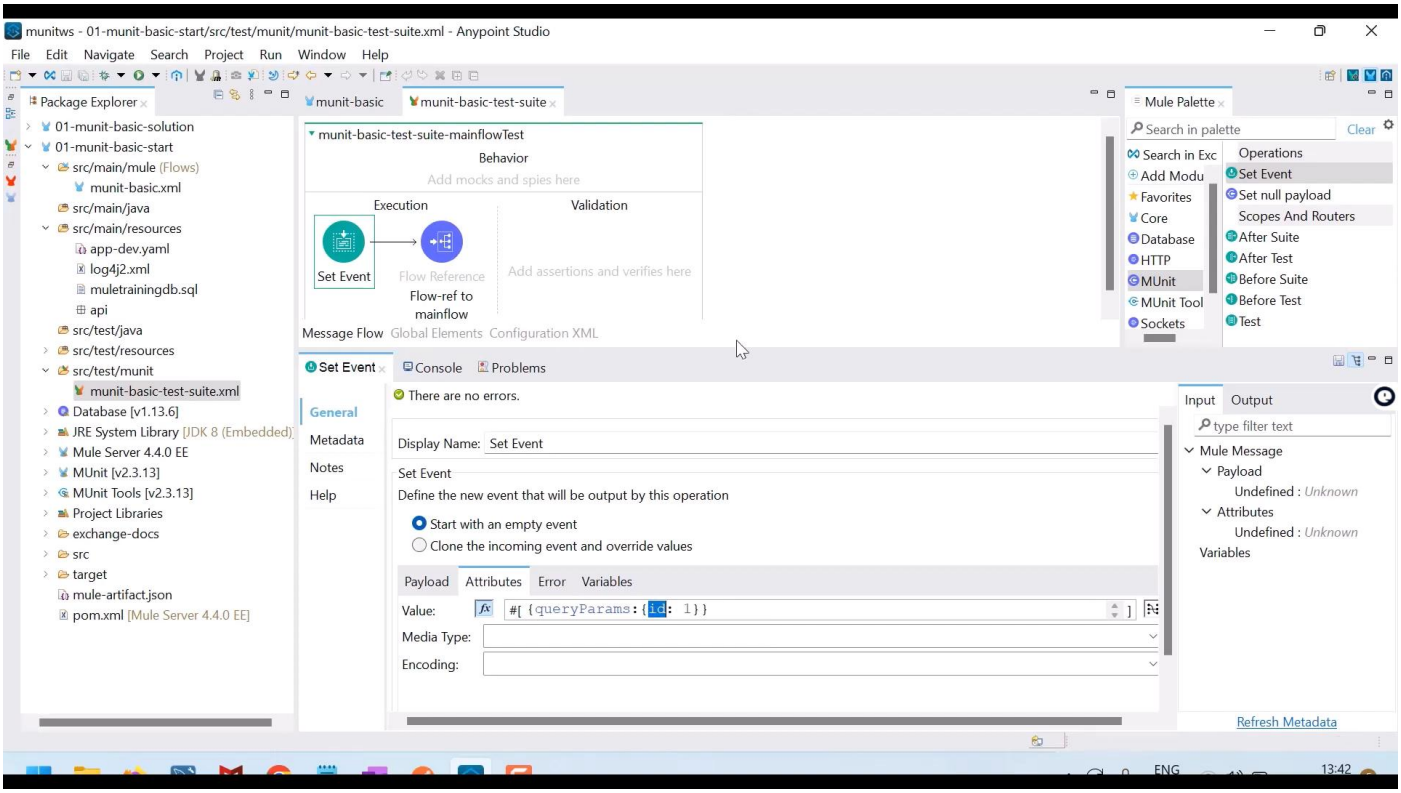
An MUnit test is the basic processor of an MUnit test suite and it represents the scenario to test.

Three optional scopes divide MUnit tests:

Scope	Description
Behavior	The behavior scope sets all the preconditions before executing the test logic.
Execution	The execution scope contains the testing logic which waits for all processes to finish before executing the next scope.
Validation	The validation scope contains all the validations for the results of the execution scope.

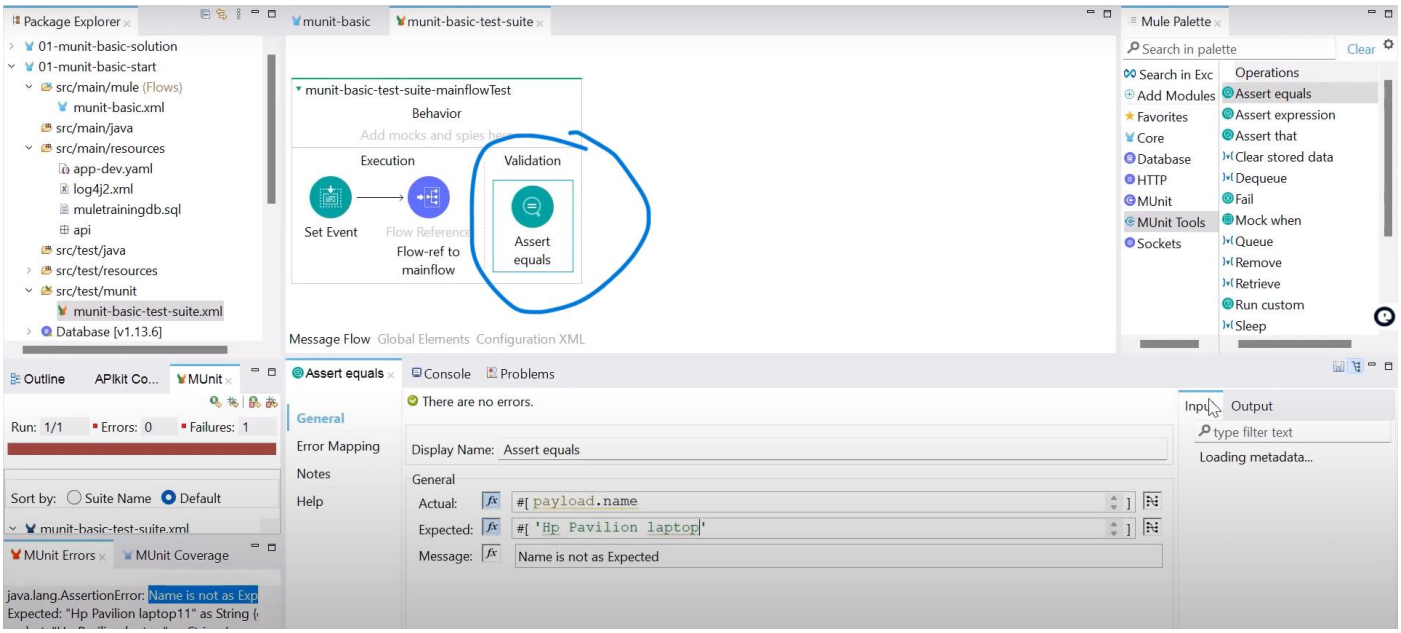
>> First we need to set mule event, which contains payload, attributes, error, variables.

In our case we need to put queryparams, we can put that in attributes.



We need to validate the test cases, a simple test cases below, assert equals to product by brand name.

Assert equal:



Actual : payload.name

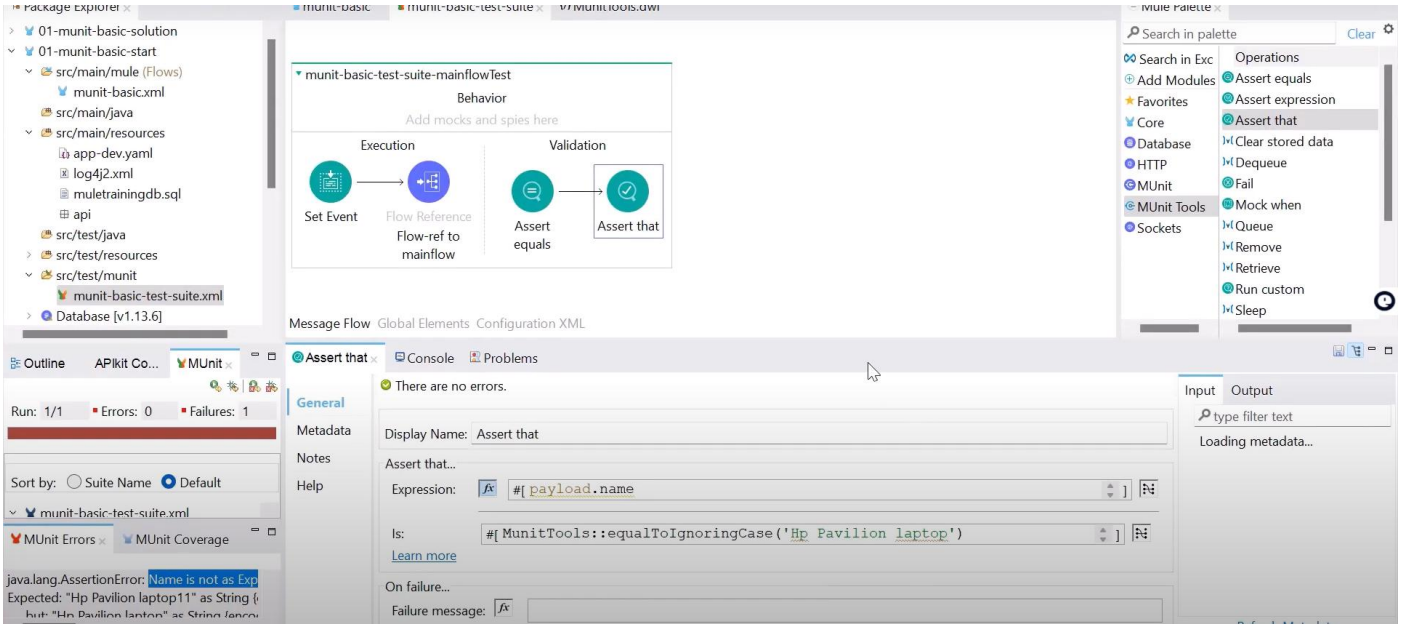
>> payload will be an array which contains details of product by brand

Expected: our expected output

>> if it matches with actual output, then test cases will pass or else it fails.

Message: if the test is fail, then this message will be seen at logs.

Assert that:



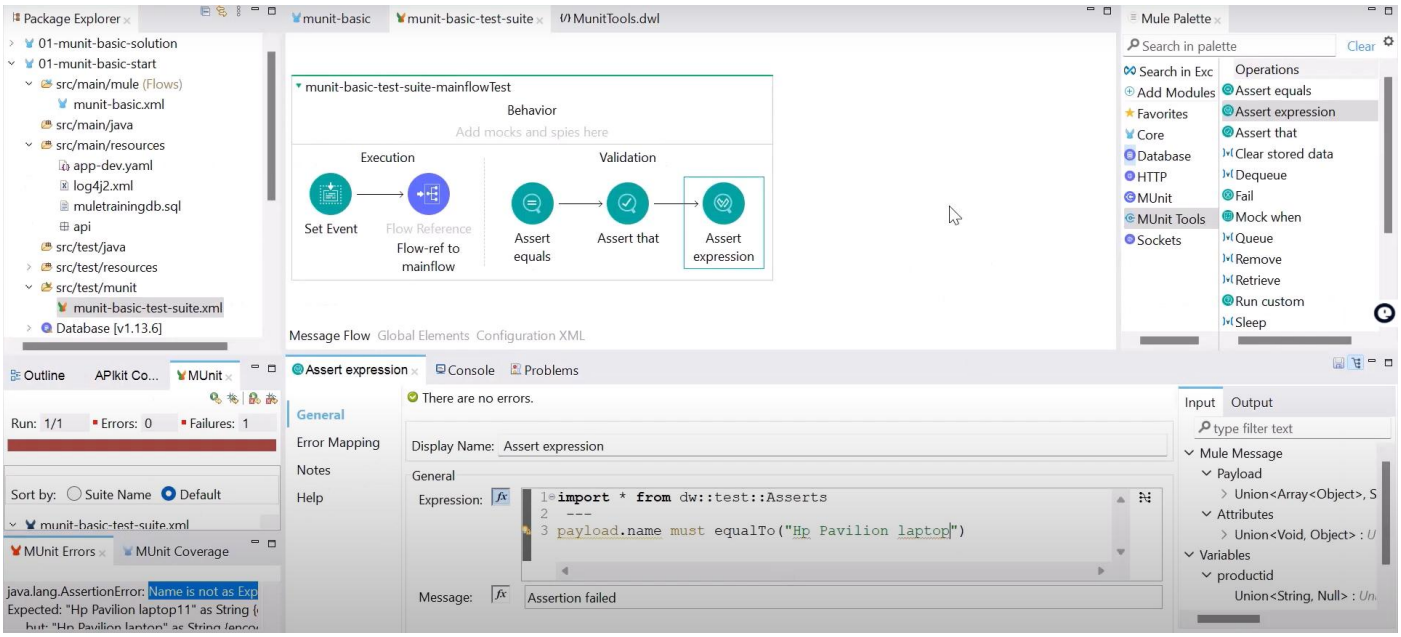
>> Assert that link: https://docs.mulesoft.com/munit/latest/assertion-event-processor

Matchers link: https://docs.mulesoft.com/munit/latest/munit-matchers

Assert Expression:

For big expression, we use assert expression

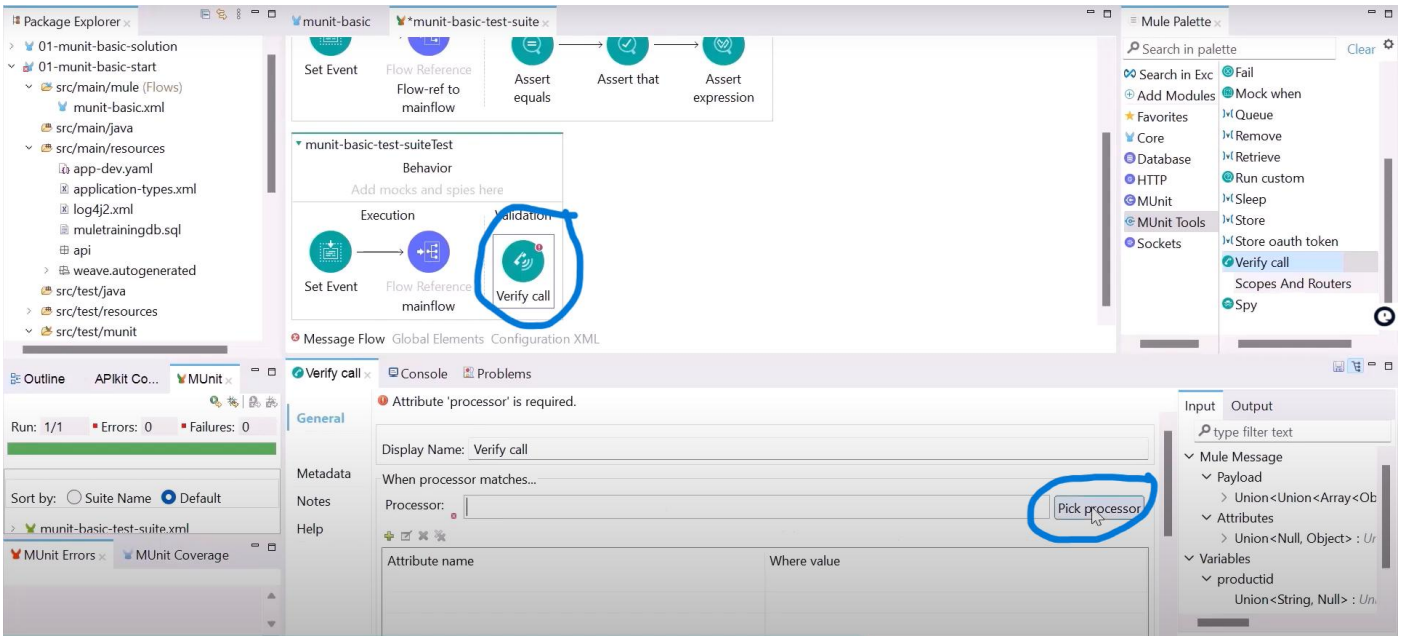
Link : https://docs.mulesoft.com/munit/latest/dataweave-assertions-library



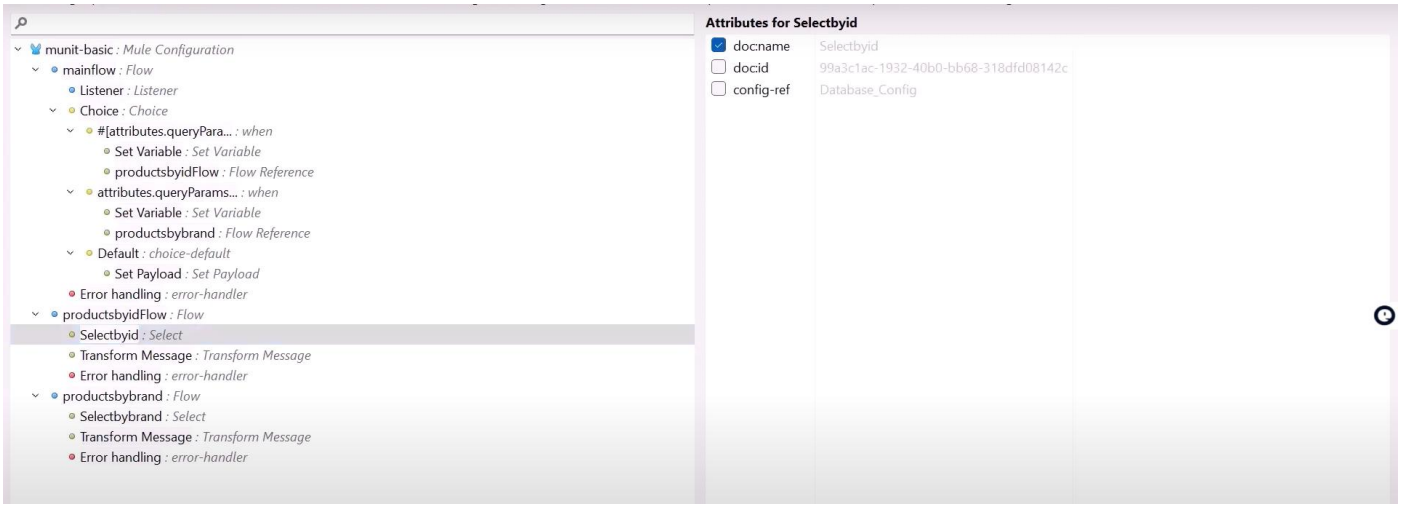
Example:

Write a functional test, whenever I am passing a queryParam as product by id, I want to verify that call is going to particular database or not.

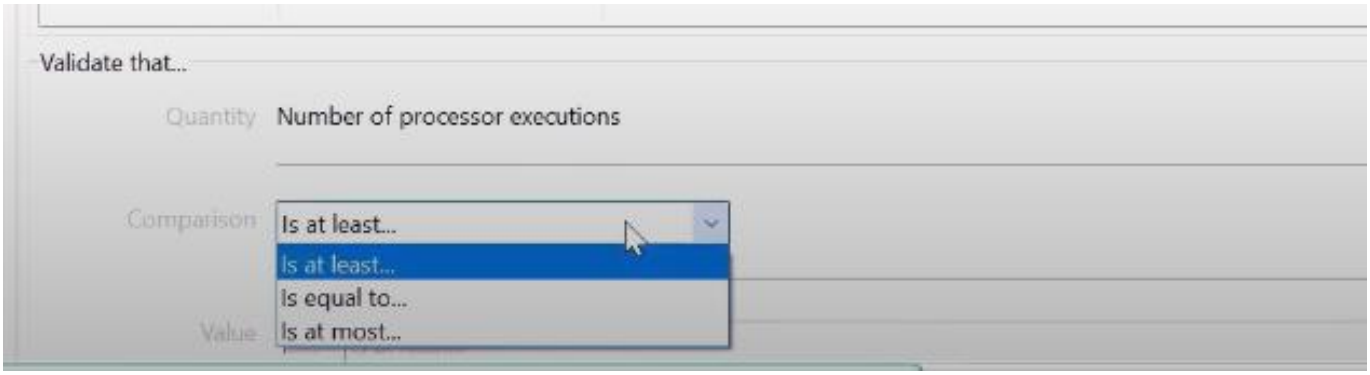
Verify call:

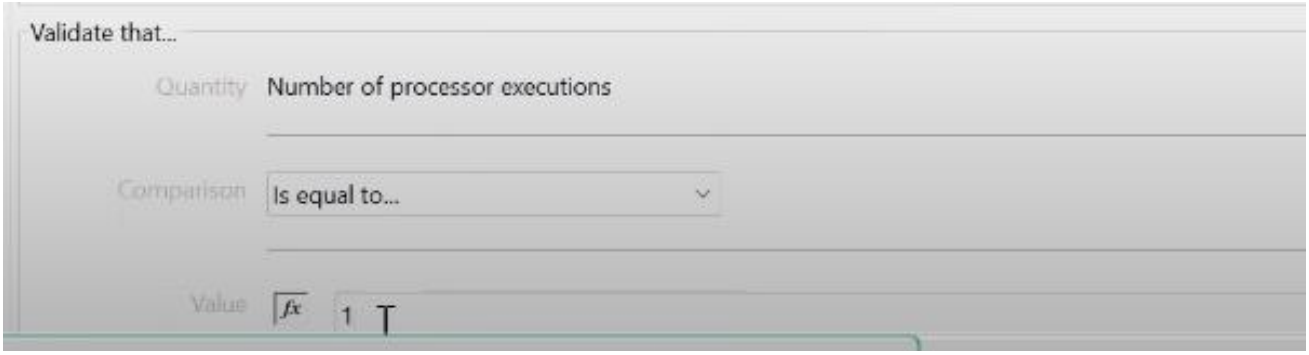


>> we need to specify for which processor to verify call for that click on pick processor



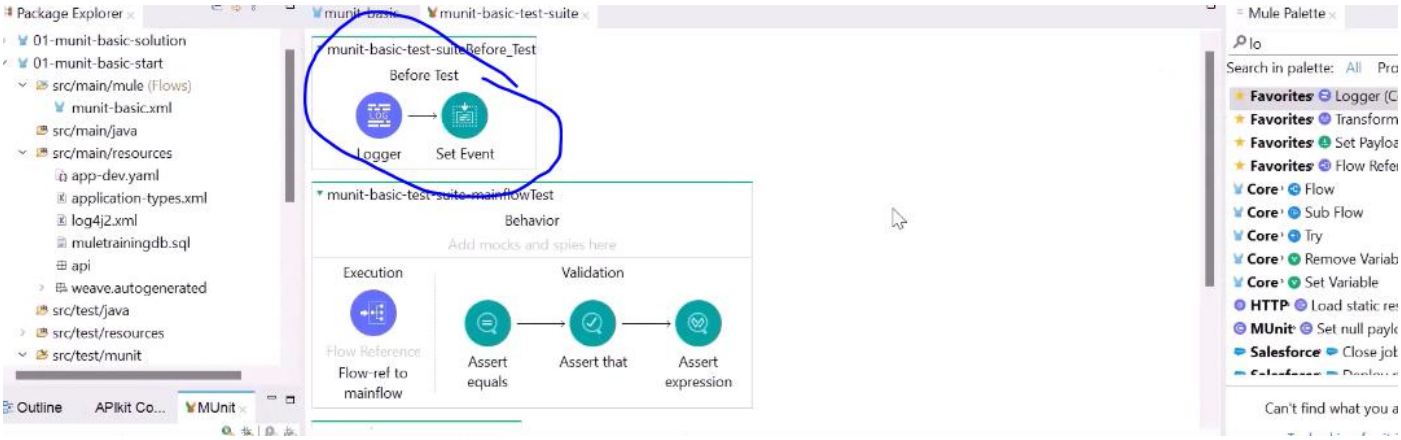
>> we can check how many times this processor went through





Before test:

It will execute before test, if we have same set events, we can place that in before test.

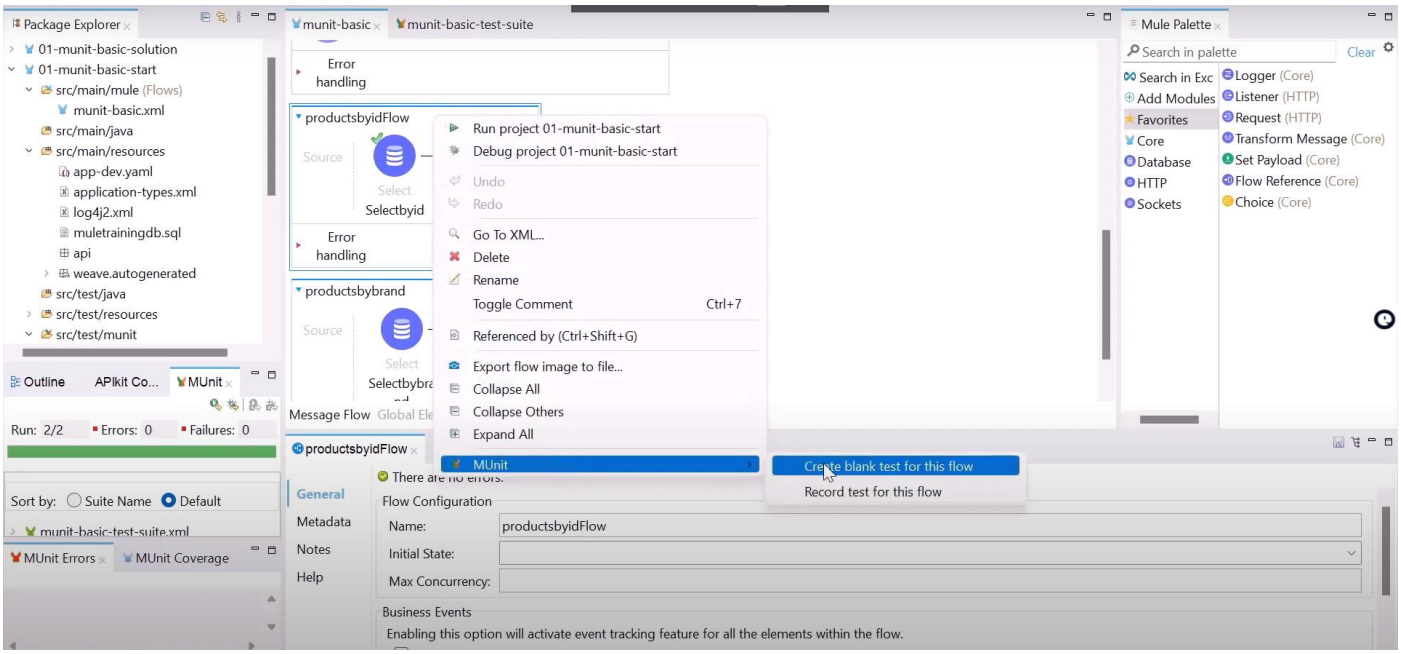


Unit test case:

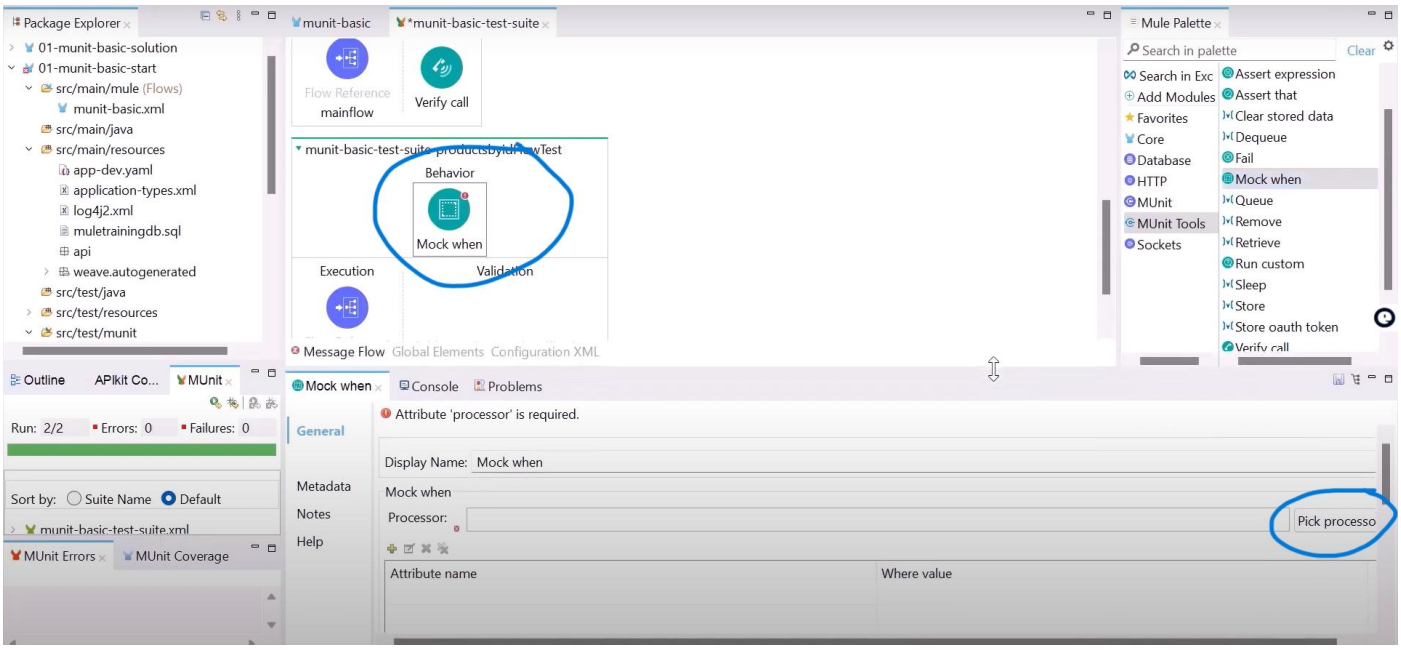
A unit test case in MuleSoft using MUnit is a specific, isolated test that verifies a small, functional unit of code within a flow.

>> we need to mock all the external calls.

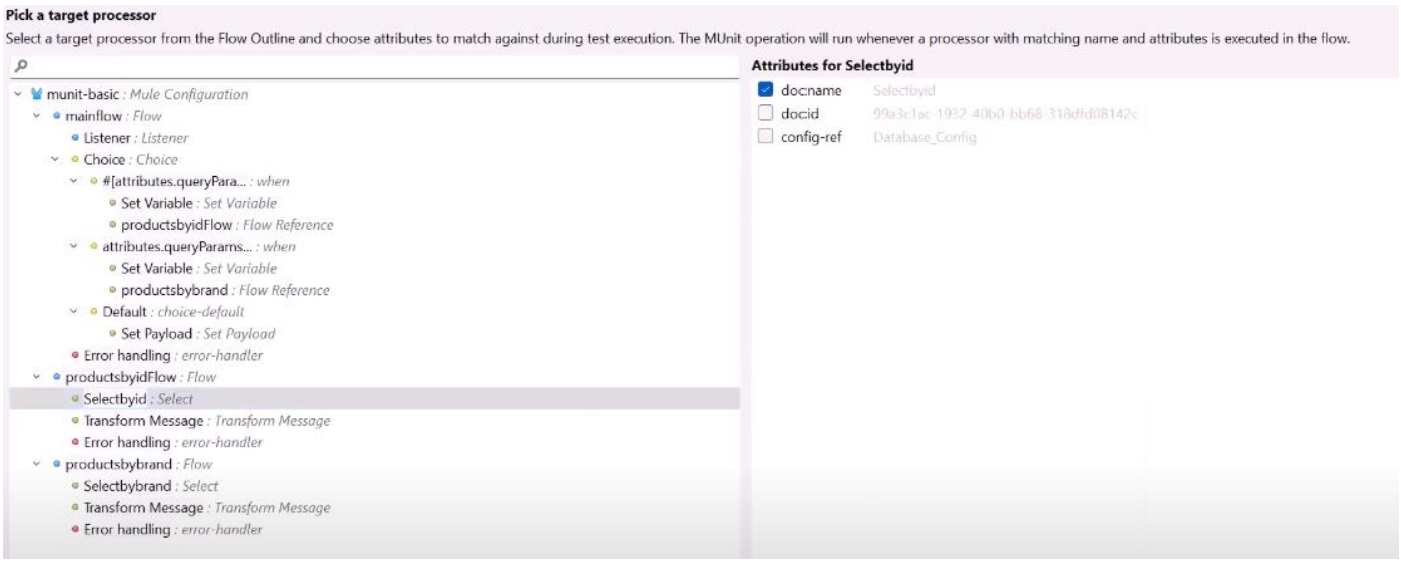
>> right click on flow, which we want to mock > Munit > create blank test



We know the input and output of database



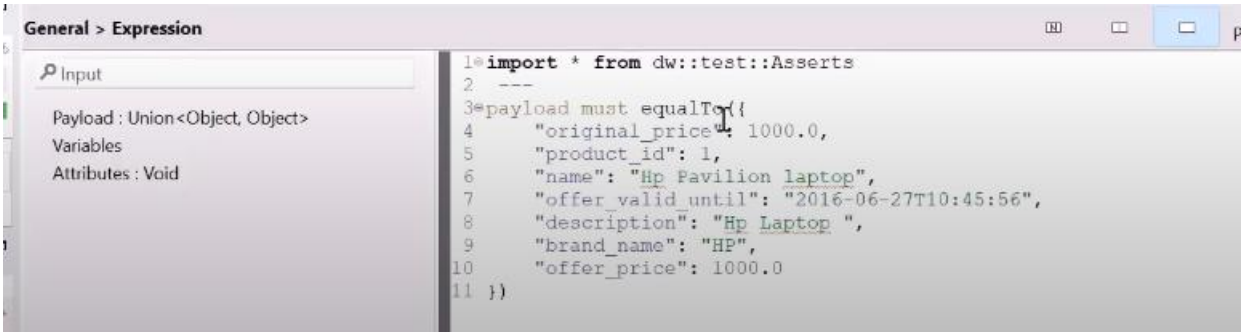
Click on pick processor



What should the mock return

>> we know the mock return will be an array as shown below, we need to paste this in value tab.

>> then in validation tab put assert expression must equal to ()

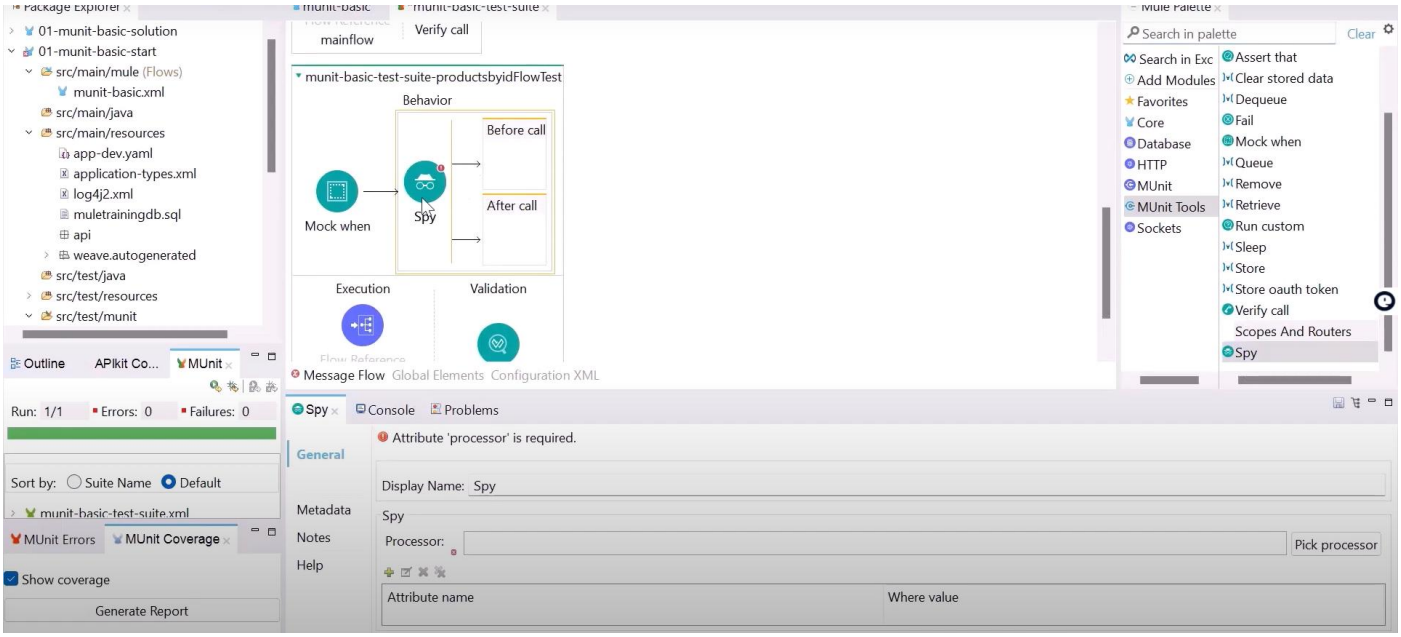


Spy:

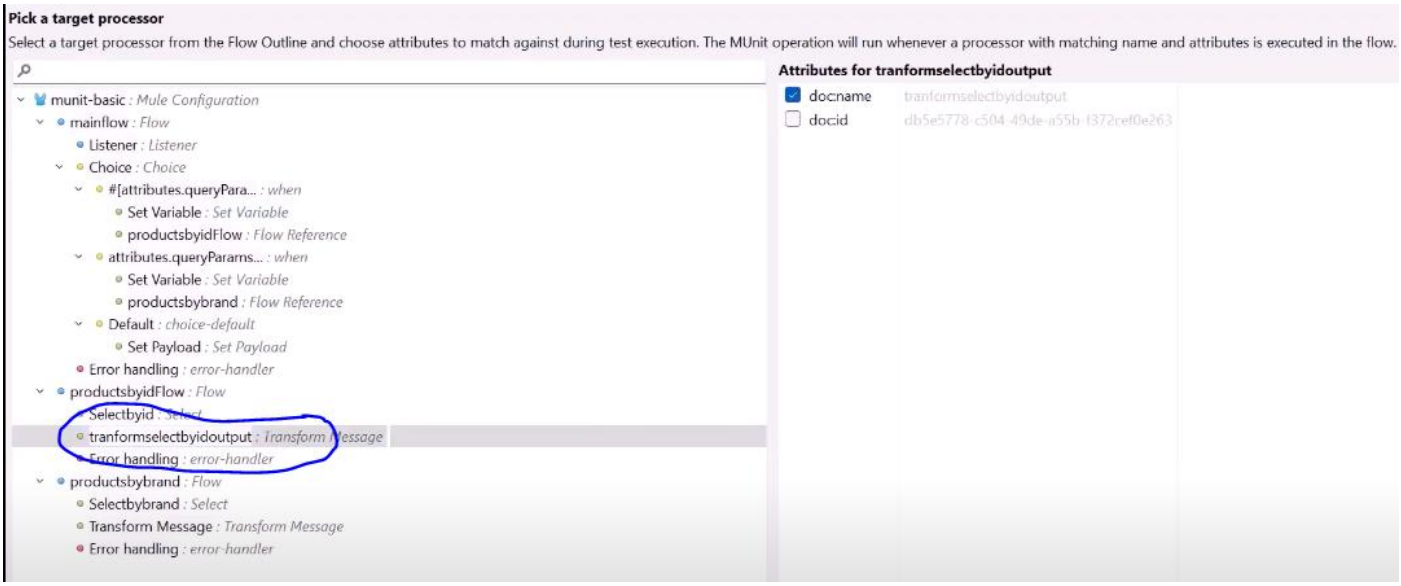
The Spy Processor enables you to spy on what happens before and after an event processor is called.

This enables you to validate, for example that a selected Mule Event reaches a specific event processor containing a specific payload or variable.

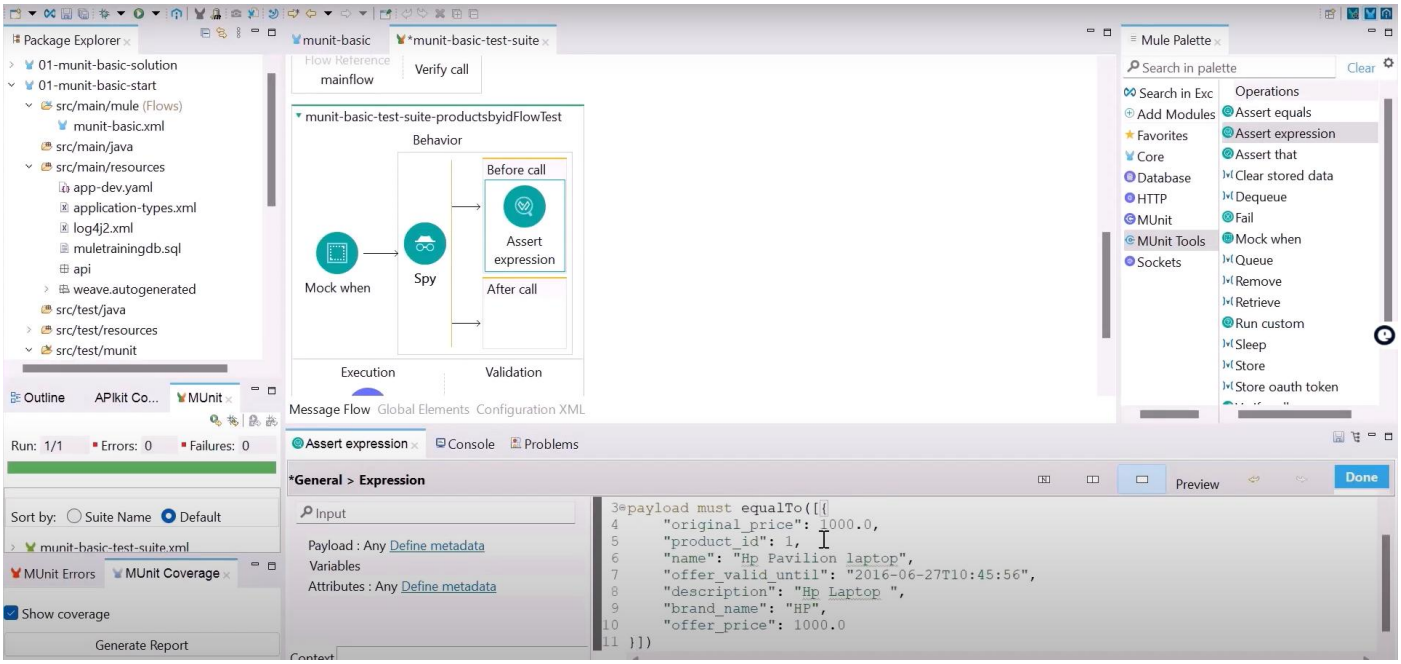
Setting a spy processor tells MUnit to run a set of instructions (usually assertions or verifications) before and/or after the execution of the spied event processor.



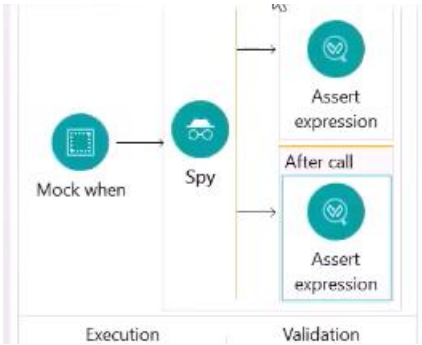
>> Pick processor which we want to spy



Before transform message, I am expecting payload to eqaul to array. As shown below

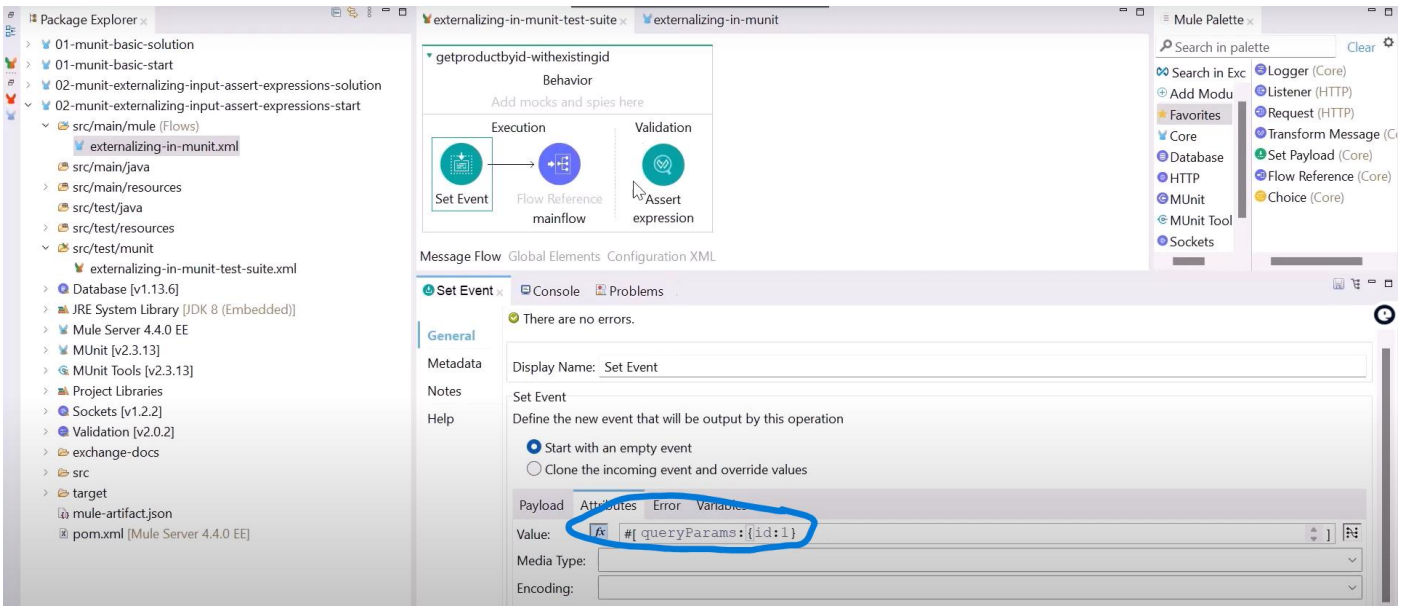


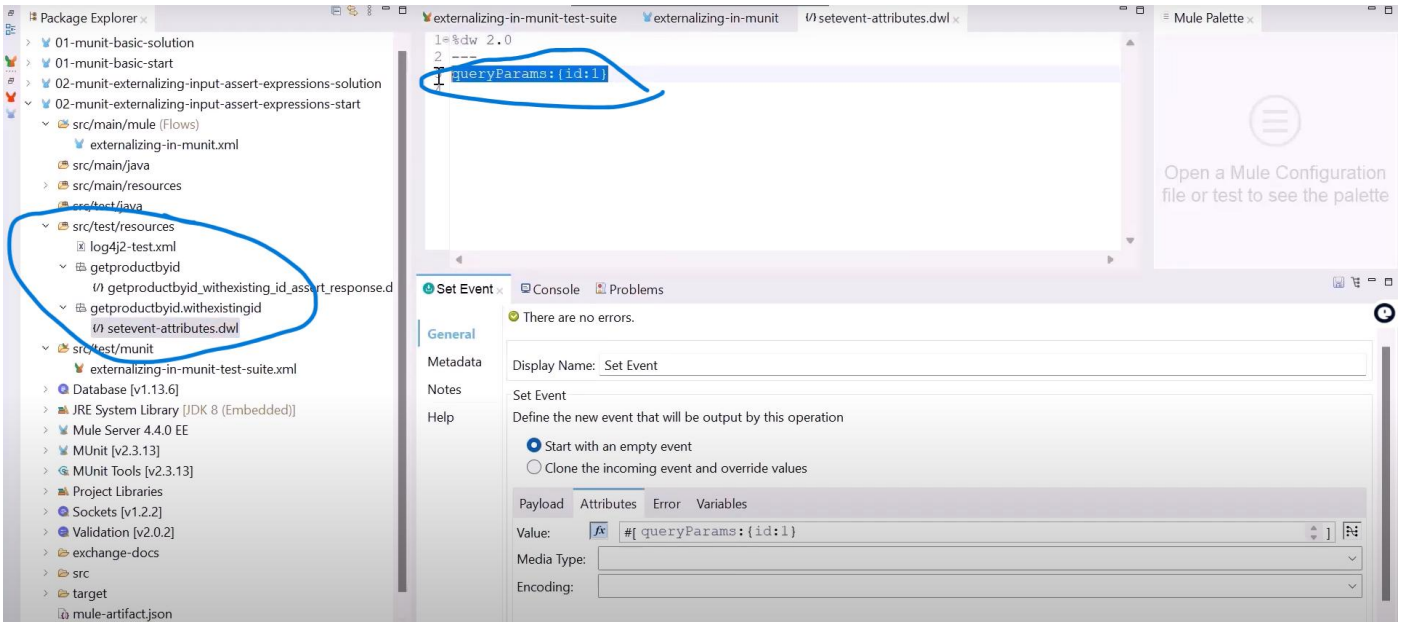
>> and after transfor message



Best practise:

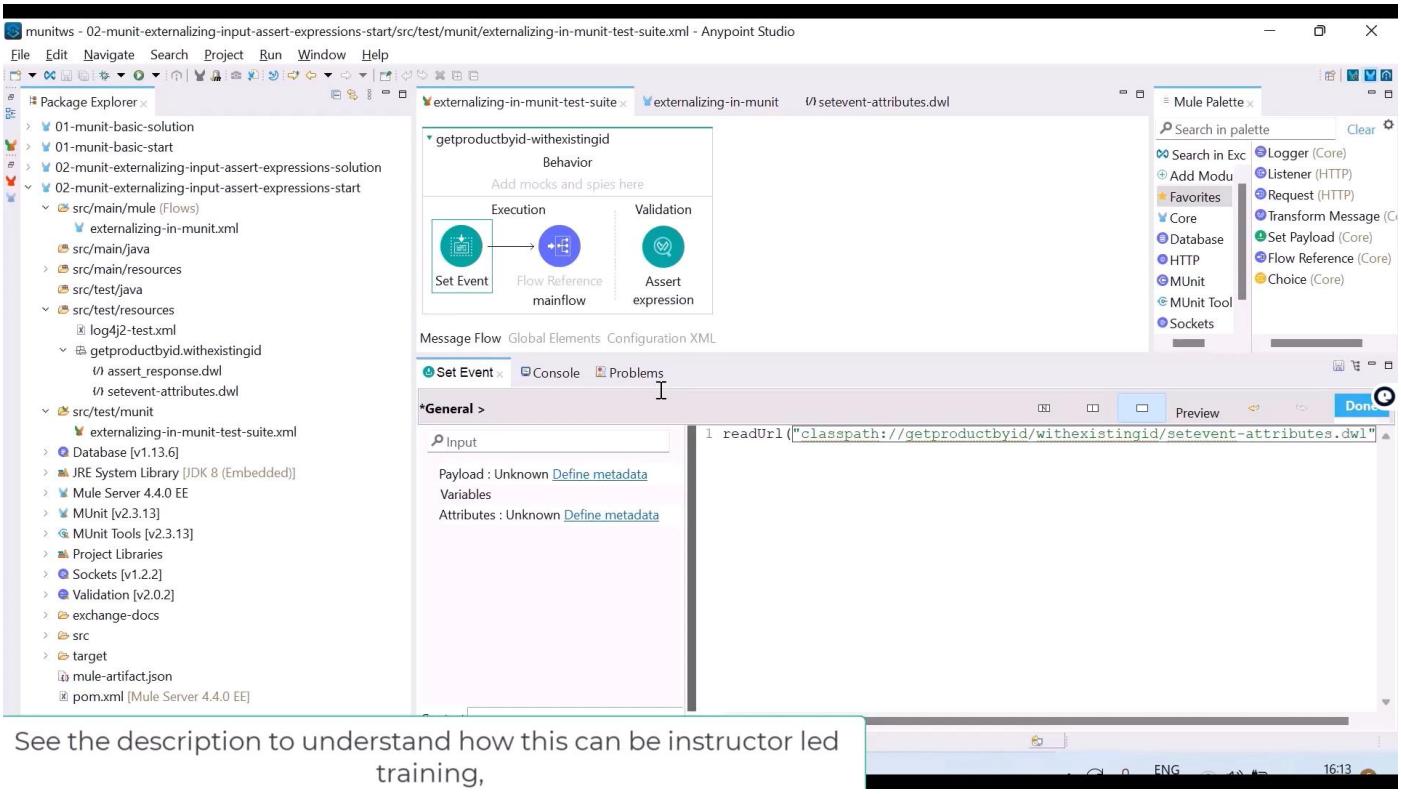
If we want to change queryParam value in future, it is not best way to put it here, we need to create separate dwl file under src/test/resources.



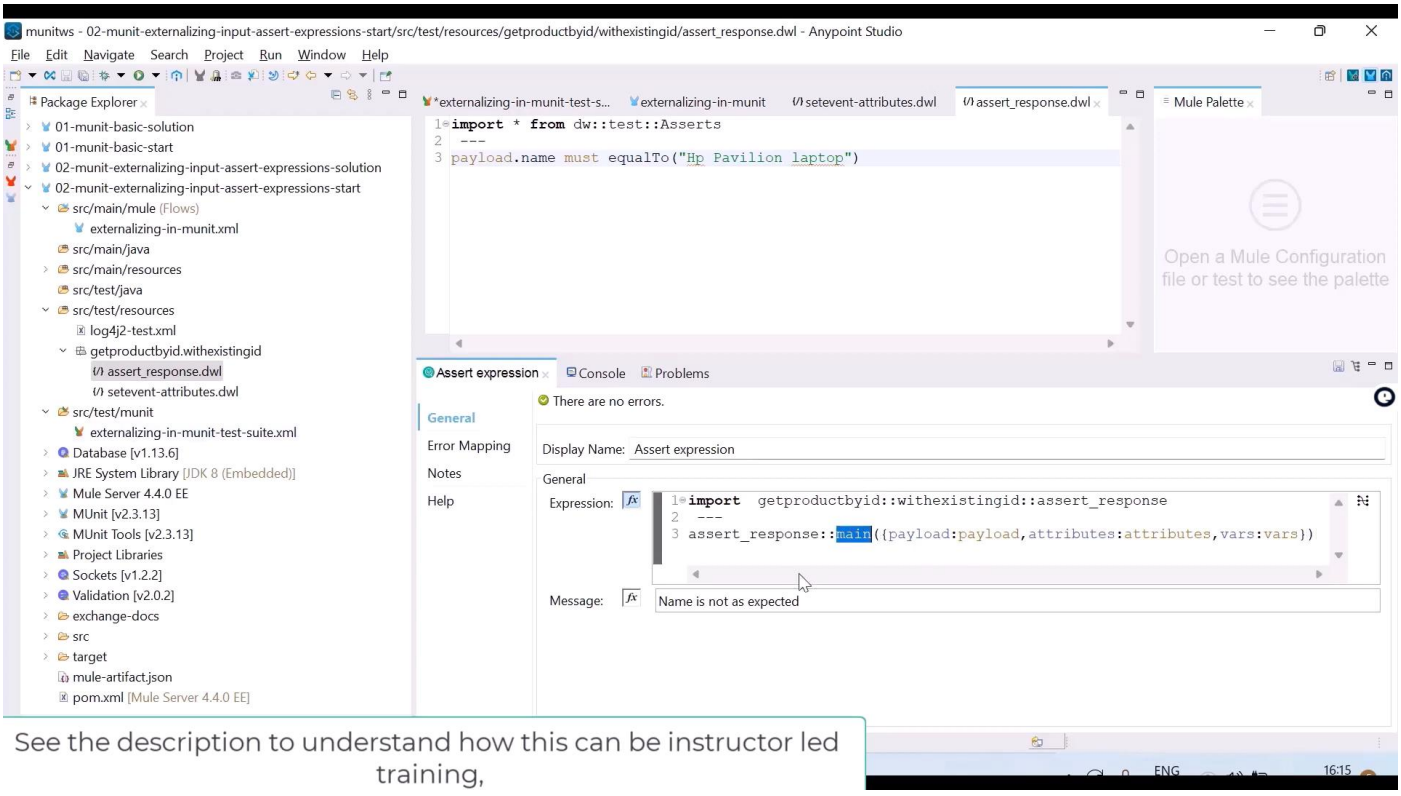


How to refer these files

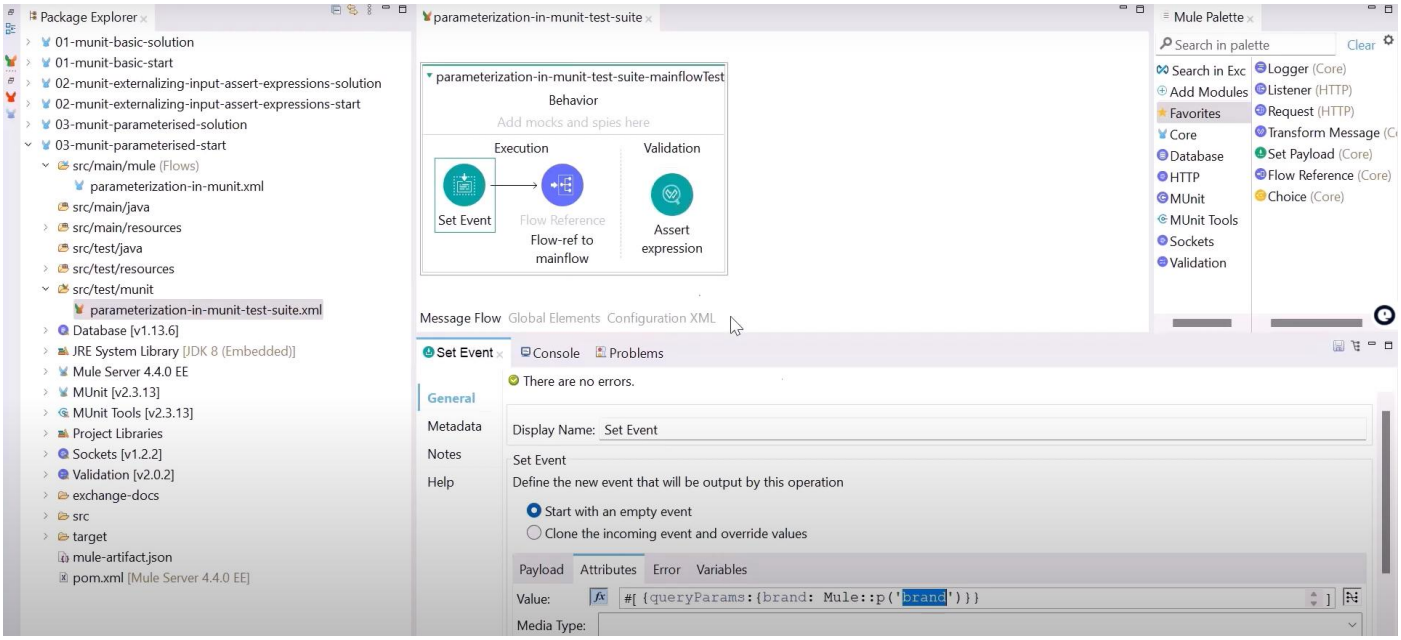
For set event

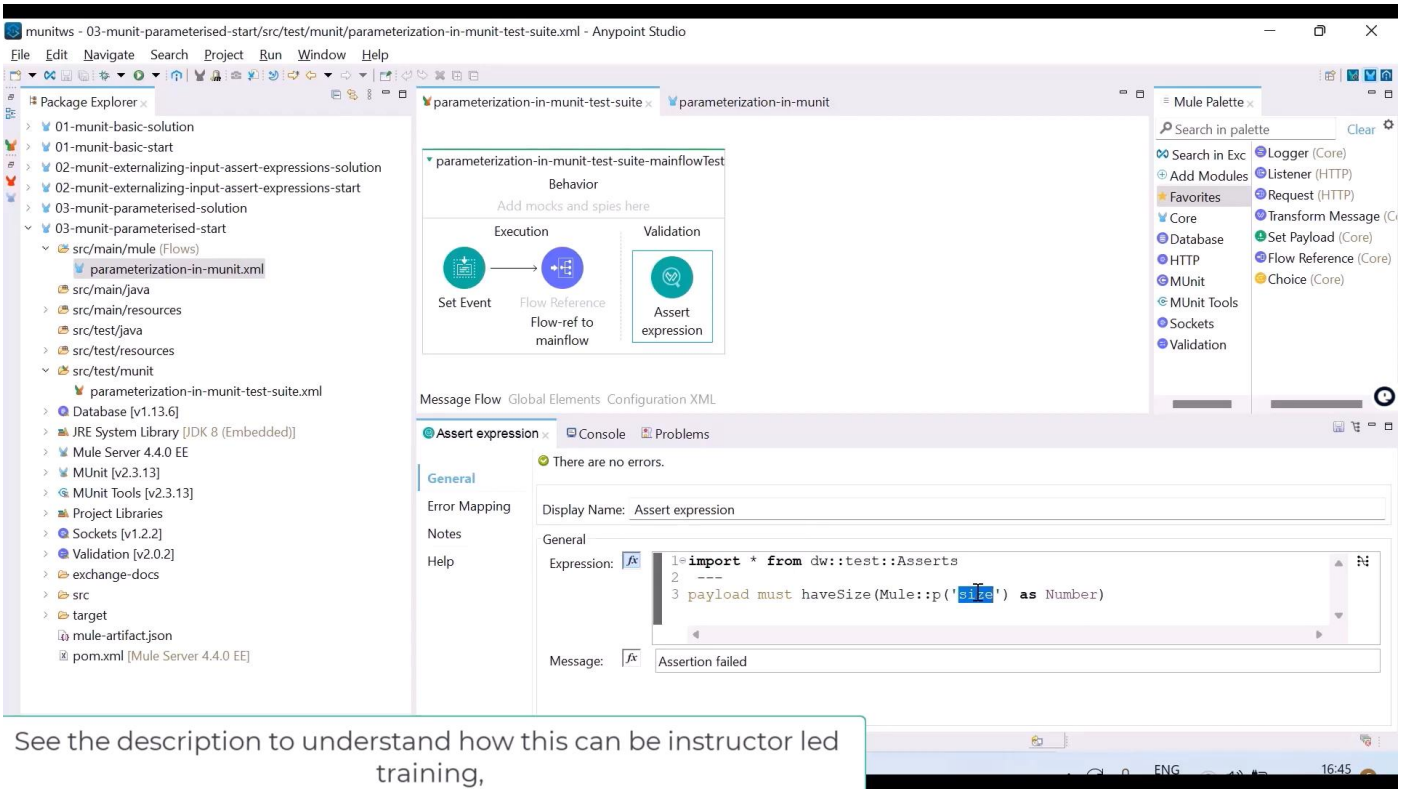


For assert expression



>> instead of hardcoding values into set event and assert expressiom, we can write like this, as shown below.

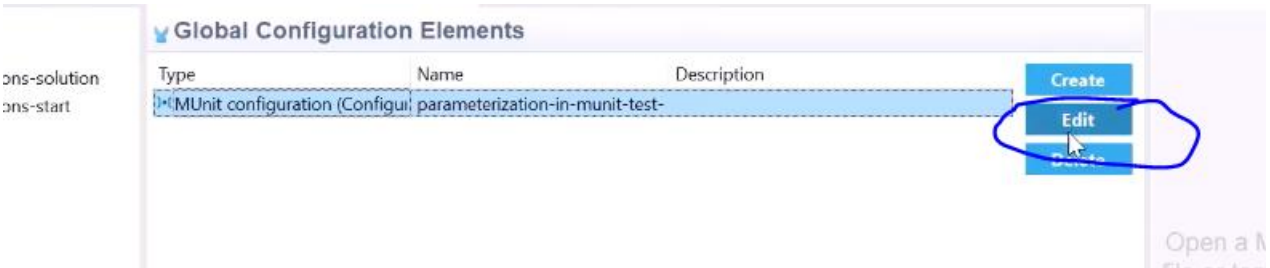




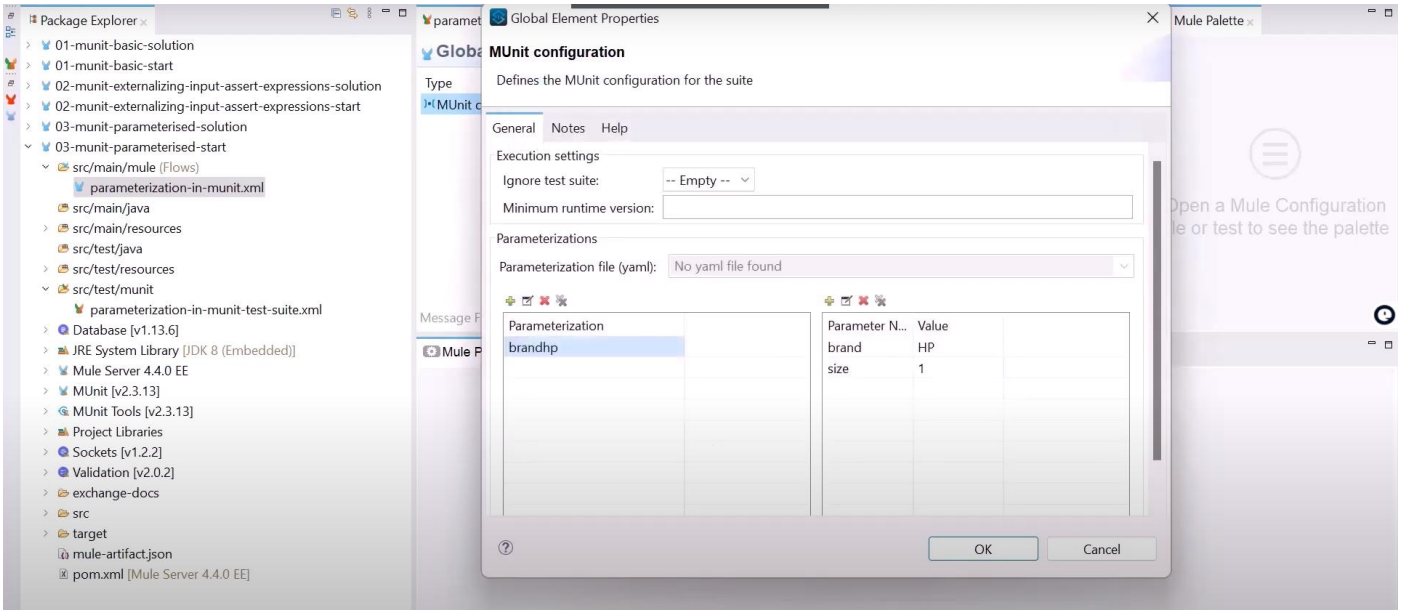
See the description to understand how this can be instructor led training,

How to pass these parameters to test

Go to global elements > edit



Click on + symbol and add parameters and values



Test cases for Error handling:

We know the input and output

>> create dwl files and refer them to set event, assert expression.